Memorando

Aplicações Móveis, 2026/2027

De: Willfredy Vieira Dias (Engenharia Informática, ISPTEC)

Nº de Matrícula: 20200204

Assunto: Laboratório 3 – Parte 1: Componentes da Aplicação Android (Activity, Service, Broadcast Receiver e Content Provider)

Data: 10/11/2026

1. Introdução

Este laboratório estuda os componentes de uma aplicação Android: actividades, serviços, broadcast receivers e content providers, e o ciclo de vida da actividade (back stack e tarefas). Fiz tudo num único projecto (pacote ao.co.isptec.aplm.lab3) em Kotlin com Jetpack Compose, em vez de XML, porque foi a forma usada no Lab2. Por isso o botão "Encerrar" chama o método finishButtonPressed() pelo onClick do Compose, e não pelo atributo android:onClick. Os projectos ActivityLifecycle.zip e ServiceSimple.zip foram substituídos por código equivalente escrito por mim.

2. Experiências Realizadas

Exercício I – Actividades

1. Botão Encerrar: o botão chama finishButtonPressed(), que chama finish(). A actividade termina e a aplicação fecha.

```
private fun finishButtonPressed() {
    finish()
}
```

2. Mensagens de log: cada callback do ciclo de vida escreve uma mensagem com Log.i("MainActivity", ...).

```
override fun onResume() {
    super.onResume()
    Log.i(TAG, "onResume")
    Toast.makeText(this, "onResume", Toast.LENGTH_SHORT).show()
}
```

```
I MainActivity: onCreate
I MainActivity: onStart
I MainActivity: onResume
I MainActivity: onPause
I MainActivity: onStop
I MainActivity: onCreate
I MainActivity: onStart
I MainActivity: onResume
I MainActivity: onPause
I MainActivity: onStop
```

```
I MainActivity: onRestart
I MainActivity: onStart
I MainActivity: onResume
I MainActivity: onPause
I MainActivity: onStop
```

Figura 1 – Logcat real: abrir, Home, voltar a abrir, retomar e sair.

a) Ao abrir a aplicação aparece: onCreate → onStart → onResume. A actividade é criada, fica visível e depois fica em primeiro plano.

b) Ao pressionar "Encerrar": onPause → onStop → onDestroy. O finish() tira a actividade da back stack e ela é destruída.

c) Ao pressionar "Home": onPause → onStop. A actividade deixa de estar visível mas não é destruída; continua na back stack da tarefa.

d) Ao retomar pela lista de aplicações recentes: onRestart → onStart → onResume. Como a actividade só estava parada, o sistema não chama onCreate; reinicia-a e volta a pô-la em primeiro plano.

e) Com o Toast no onResume, aparece a mensagem "onResume" no ecrã sempre que a actividade volta ao primeiro plano (ao abrir e ao regressar do Home).

Figura 2 – Toast "onResume" no emulador (inserir captura).

3. Navegação entre actividades: ao abrir uma actividade nova (ex.: Bloco de notas), a anterior recebe onPause e onStop, e a nova recebe onCreate, onStart e onResume, ficando no topo da back stack. Ao carregar em VOLTAR, a actividade do topo é destruída (onPause, onStop, onDestroy) e a anterior faz onRestart, onStart e onResume. O wireframe das actividades está na secção 3.1.

Exercício II – Serviços

1. Ao pressionar "Start Service" e depois "Stop Service" aparecem os toasts: "Service created" (onCreate, só na primeira vez), "Service started: N" (onStartCommand, em cada startService) e "Service destroyed" (onDestroy, com stopService). Se o serviço já estiver a correr, um novo startService não chama onCreate, só onStartCommand.

2. Estado partilhado: criei a classe Lab3App (extends Application) com um contador. A actividade incrementa o contador antes de chamar startService, e o serviço lê o valor no onStartCommand. Como o objecto Application é único para toda a aplicação, a actividade e o serviço vêem o mesmo valor.

```
private fun startSimpleService() {
    (application as Lab3App).incrementServiceStartCount()
    startService(Intent(this, SimpleService::class.java))
}

override fun onStartCommand(intent: Intent?, flags: Int, startId: Int): Int {
    val count = (application as Lab3App).serviceStartCount
```

```
    showToast("Service started: $count")
    return START_STICKY
}
```

Exercício III – Broadcast Receivers

Criei o MyPhoneReceiver, registado no manifesto com o filtro android.intent.action.PHONE_STATE e as permissões READ_PHONE_STATE e READ_CALL_LOG. A permissão é pedida em tempo de execução pelo botão "Pedir permissão de telefone". No onReceive leio o estado (EXTRA_STATE: RINGING, OFFHOOK, IDLE) e o número (EXTRA_INCOMING_NUMBER, só preenchido se a permissão READ_CALL_LOG for concedida) e escrevo no Logcat.

```
<receiver android:name=".MyPhoneReceiver" android:exported="true">
    <intent-filter>
        <action android:name="android.intent.action.PHONE_STATE" />
    </intent-filter>
</receiver>

override fun onReceive(context: Context, intent: Intent) {
    val state = intent.getStringExtra(TelephonyManager.EXTRA_STATE)
    val number = intent.getStringExtra(TelephonyManager.EXTRA_INCOMING_NUMBER)
    Log.i(TAG, "Estado: $state, Numero: $number")
}
```

A chamada foi simulada nos Extended Controls do emulador (Phone → Call device). No Logcat aparece "Estado: RINGING" e, ao desligar, "Estado: IDLE".

3. Desafios

O desafio é um bloco de notas simples com três actividades: ListNotesActivity, CreateNoteActivity e ReadNoteActivity, acedido pelo botão "Bloco de notas" da MainActivity.

3.1. Desenho da interface móvel

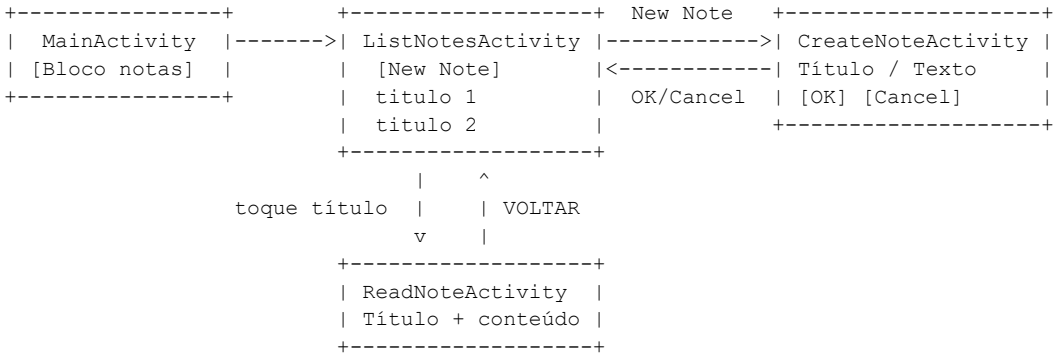


Figura 2 – Wireframe de actividades do bloco de notas.

A ListNotesActivity mostra os títulos e o botão New Note. New Note abre a CreateNoteActivity; OK guarda a nota e fecha, Cancel fecha sem guardar. Tocar num título abre a ReadNoteActivity com o título e o conteúdo; VOLTAR regressa à lista.

3.2. Detalhes de implementação

Estrutura de dados: cada nota é um data class Note(title, content). As notas ficam numa MutableList<Note> dentro da classe Lab3App (Application), partilhada pelas três actividades. Os dados vivem em memória enquanto o processo existir.

```
data class Note(val title: String, val content: String)

class Lab3App : Application() {
    val notes = mutableListOf<Note>()
}
```

Tipos de eventos: clique no botão New Note (startActivity com Intent explícito); clique em OK (adiciona a nota à lista e chama finish()); clique em Cancel (finish() sem guardar); clique num título da lista (Intent com o índice da nota no extra note_index); botão VOLTAR (o sistema destrói a ReadNoteActivity); onResume da ListNotesActivity (recarrega os títulos para a lista aparecer actualizada depois de criar uma nota).

Comportamento: ao voltar da CreateNoteActivity, a ListNotesActivity passa por onRestart/onStart/onResume e mostra o novo título. Uma nota sem título não é guardada. A lista é um LazyColumn e os títulos ficam num mutableStateListOf, por isso o Compose redesenha a lista quando muda.

4. Referências Bibliográficas

- Android Developers – Introduction to activities: https://developer.android.com/guide/components/activities/intro-activities
- Android Developers – The activity lifecycle: https://developer.android.com/guide/components/activities/activity-lifecycle
- Android Developers – Services overview: https://developer.android.com/guide/components/services
- Android Developers – BroadcastReceiver: https://developer.android.com/reference/android/content/BroadcastReceiver
- Android Developers – TelephonyManager.ACTION_PHONE_STATE_CHANGED: https://developer.android.com/reference/android/telephony/TelephonyManager
- Android Developers – Content providers: https://developer.android.com/guide/topics/providers/content-providers
- Enunciado do Laboratório 3, Aplicações Móveis, ISPTEC 2025/26.

5. Repositório GitHub

https://github.com/KazenWilver/lab3-aplm (convite de colaborador enviado ao utilizador joaojdacosta).